PEMROGRAMAN VISUAL

Mini Project Aplikasi Penjadwalan Aktivitas

“AuraPlan”

Disusun Oleh:

Nama : Nurhayati Ningsih

NIM : F1D02410085

Kelas : C

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MATARAM

2026

1. Deskripsi Aplikasi

AuraPlan adalah sebuah aplikasi desktop berbasis *Graphical User Interface* (GUI) yang dirancang khusus untuk membantu pengguna dalam mengolola dan menjadwalkan berbagai aktivitas harian. Dalam kehidupan mahasiswa atau pekerja yang sangat padat dan terkadang juga aktivitas penting mudah lupa dilakukan atau bentrok waktunya. AuraPlan hadir sebagai solusi digital untuk mencatat aktivitas secara terstruktur, lengkap dengan detail waktu, prioritas, hingga status penyelesaian aktivitas tersebut.

Aplikasi AuraPlan ini dibangun menggunakan bahasa pemrograman Python dengan memanfaatkan library PySide6 sebagai framework utama untuk membangun antarmuka grafisnya. Keunggulan utama dari sistem ini terletak pada penyimpanan datanya, pengguna tidak perlu takut kehilangan data karena seluruh jadwal yang diinput akan tersimpan secara permanen di dalam database lokal menggunakan SQLite, tanpa memerlukan koneksi internet atau server database terpisah.

2. Tujuan *Project*

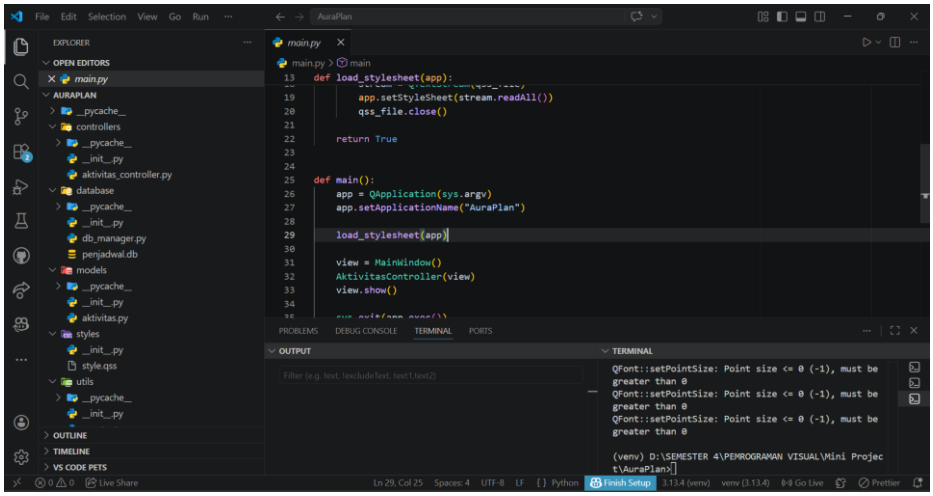
Tujuan utama dari pengembangan aplikasi AuraPlan ini adalah:

1. Membangun sebuah sistem manajemen aktivitas yang kreatif, interaktif, dan mudah digunakan oleh pengguna akhir.
2. Menerapkan arsitektur *Separation of Concerns* (SoC) secara disiplin agar kode program mudah dalam pemeliharaan, tidak saling campur aduk antara logika bisnis dan tampilan.
3. Menerapkan operasi CRUD (*Create, Read, Update, Delete*) dengan database SQLite secara *seamless*.
4. Mengembangkan pengalaman pengguna yang baik melalui penerapan *styling* eksternal (QSS), validasi input, serta penanganan *error* yang ramah.
5. Mengimplementasikan materi pemrograman visual yaitu PySide yang telah dipelajari.

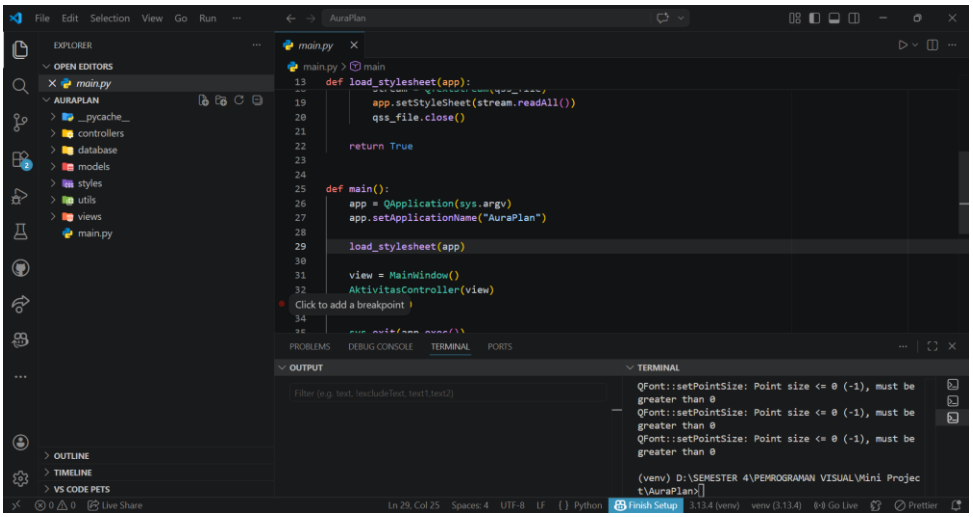
3. Struktur *Project* dan Penerapan *Separation of Concerns* (SoC)

Separation of Concerns (SoC) adalah prinsip desain perangkat lunak di mana sebuah program dibagi menjadi beberapa bagian yang maisng-masing memiliki tanggung jawab spesifik. Pada AuraPlan, arsitektur SoC diterapkan dengan memisahkan kode ke dalam empat lapisan utama yaitu *View*, *Controller*, Model, dan Database. Alur data pada pada sistem ini bersifat satu arah dan terisolasi. Misalnya, komponen view seperti `QpushButton` tidak pernah tahu dari mana datanya berasal, ia hanya menyampaikan signal jika tombol yang diklik. *Controller* yang menangkap signal tersebut, meminta data dari database, memprosesnya lalu memerintahkan *view* untuk memperbarui tampilannya.

Berikut adalah struktur folder dari keseluruhan *source* code aplikasi AuraPlan:



Berdasarkan gambar di atas, aplikasi AuraPlan terlihat memiliki folder utama yang masing-masing memiliki tugas spesifik sesuai prinsip *Separation of Concerns* (SoC) yaitu database, *models*, *views*, *controller*, *utils*, dan *styles*. Hal kecil namun krusial yang perlu diperhatikan adalah keberedaan file `_init_.py` berupa file kosong didalam setiap folder. File ini merupakan syarat mutlak di Python agar folder tersebut dikenali sebagai package yang sah dan dapat di *import* oleh modul lainnya. Jika file ini di hapus, maka perintah seperti `from views.main_windows import MainWindow` akan menimbukan *error*.



Berdasarkan gambar diatas, folder `views` bertugas murni membangun elemen antarmuka visual tanpa mencampurkan logika bisnis atau query `database`. Di sisi lain, folder `controllers` berperan sebagai penghubung utama yang menangkap signal dari view untuk memproses alur logika aplikasi, memvalidasi data melalui models hingga mengirim perintah penyimpanan ke `database`. Sementara itu, fungsi pendukung seperti penyimpanan konstanta teks dipisahkan ke dalam `utils`, dan seluruh desain antarmuka dikemas secara terpusat dalam file `style.qss` di folder `style.py`, sehingga menciptakan struktur kode yang modular, rapi, dan mudah dikembangkan tanpa saling tergantung antar modul.

4. Desain Database

a. Pemilihan sistem database

AuraPlan menggunakan sistem manajemen database relasional (RDBMS) yaitu SQLite. SQLite dipilih karena bersifat *serverless* (tidak memerlukan layanan background yang berjalan terpisah), ringan, dan datanya tersimpan dalam satu file lokal bernama `penjadwalan.db`. Hal ini sangat ideal untuk aplikasi desktop berskala personal/user tunggal.

b. Skema tabel aktivitas

Tabel utama dalam database AuraPlan diberi nama aktivitas yang terdiri dari sepuluh kolom. Kolom pertama adalah `id` dengan tipe data `INTERGER` yang berperan sebagai *primary key* dengan atribut `AUTOINCREMENT` untuk menciptakan ID unik secara otomatis. Selanjutnya, terdapat kolom nama_aktivitas bertipe `TEXT` dengan constraint `NOT NULL` untuk menyimpan nama kegiatan. Untuk menangkap informasi waktu, digunakan tiga kolom bertipe `TEXT` dengan constraint `NOT NULL`, yaitu tanggal (format YYYY-MM-DD), waktu_mulai, dan waktu_selesai (format HH:mm). Klasifikasi dan tingkat urgensi kegiatan ditangani oleh kolom kategori dan prioritas yang keduanya bertipe `TEXT` dan `NOT NULL`. Sebagai tempat menyimpan catatan tambahan, terdapat kolom deskripsi bertipe `TEXT` dengan nilai default kosong. Sebagai fitur kreatif orisinal aplikasi, kolom status dengan tipe `TEXT` dan default 'Belum' disertakan untuk menandai apakah suatu tugas sudah diselesaikan. Terakhir, kolom created_at bertipe `TEXT` secara otomatis mereka *timestamp* kapan data tersebut pertama kali disimpan ke dalam database menggunakan fungsi default `datetime('now', 'localtime')`.

c. Keunggulan desain database

Keunggulan desain database adalah pencegahan *crash* saat *update* skema yaitu pada *file* `db_manager.py` fungsi `_init_db` menjalankan perintah `CREATE TABLE IF NOT EXISTS`. Selain itu, terdapat blok `try-except` yang menjalankan `ALTER TABLE` secara aman. Ini memastikan bahwa jika di masa depan ada penambahan kolom baru pada versi aplikasi yang lebih baru, database lama pengguna tidak akan rusak atau *error*, melainkan kolom baru akan langsung ditambahkan secara otomatis.

5. Fitur Utama dan Cara Kerjanya

5.1 Fitur utama CRUD (*Create, Read, Update, Delete*)

a. *Create* (Tambah data)

Pengguna menekan tombol "+ Tambah Aktivitas". Sistem akan memanggil `DialogAktivitas` (jendela terpisah). Terdapat 7 komponen input berbeda yaitu `QlineEdit` (nama), `QdateEdit` dengan popup kalender (tanggal), 2x `QtimeEdit` (waktu), 2x `QcomboBox` (kategori & prioritas, `QtextEdit` (deskripsi), dan `QchecBox` (status). Setelah form diisi, *Controller* melakukan validasi (waktu selesai > waktu mulai) sebelum menyurus Database menyimpannya.

b. *Read* (Tampil data)

Saat aplikasi dibuka, *Controller* membaca seluruh data menggunakan *query* SELECT*. Data mentah tanggal (YYYY-MM-DD) dikirim ke Model untuk diformat menjadi format Indonesia (misal: 15 Januari 2025) sebelum akhirnya ditampilkan secara rapi QtableWidget dengan pewarnaan kolom prioritas (Merah, Kuning, Hijau).

c. *Update* (Edit Data)

Pengguna memilih satu baris data di tabel, lalu menekan tombol "Edit". *Controller* mengambil ID baris tersebut, meminta Database mencari data spesifik berdasarkan ID, lalu mengirimkan datanya ke `DialogAktivitas` untuk diubah isinya.

d. *Delete* (Hapus Data)

Pengguna memilih data dan menekan "Hapus". Sistem menampilkan, `QMessageBox.question` (Dialog Konfirmasi) untuk mencegah penghapusan tidak sengara, jika disetujui, *query* `DELETE` dijalankan.

5.2 Penerapan *event handling*

a. Terdapat beragam signal seperti `cliked` (tombol), `triggered` (menu bar), `currentTextChanged` (dropdown filter), dan `cellDoubleClicked` (klik ganda pada tabel untuk edit cepat).

b. Pada *splashscreen* (`DialogWelcome`), jendela diatur menjadi *frameless. Event* `mousePressEvent`, `mouseMoveEvent`, dan `mouseReleaseEvent` di *override* menggunakan kordinasi global agar pengguna dapat menyeret layar *welcome* tersebut.

c. Event `keyPressEvent` di *override* Menekan tombol "Enter" di mana saja langsung memindahkan fokus kursor ke kolom pencarian. Menekan tombol "Esc" secara instan mereset seluruh filter tabel.

d. Dialog handling bawaan seperti `QfontDialog` dan `QfileDialog`, menu Edit menyediakan fitur "Ubah Font Tabel" yang membuka dialog *font native* sistem. Menu

File menyediakan fitur “Export Jadwal” yang membuka dialog penyimpanan file untuk mengekspor data ke .txt.

e. Event closeEvent di *override* pada main Windows, daat pengguna menekan tombol “X” di pojok jendela, ia memicu dialog konfirmasi untuk mencegah penutupan yang tidak disengaja.

5.3 Alur kerja sistem

a. Pengguna mengklik tombol "Simpan" di Dialog kemudian memancarkan *signal* `clicked()`.

b. Dialog menutup dirinya dan mengembalikan status *Accepted*.

c. `Controller` menangkap status tersebut, meminta data inputan dari Dialog melalui fungsi `get_data()`.

d. `Model` mengirim data tersebut ke fungsi `validate_aktivitas()` di `Model`. `Model` memeriksa apakah jam selesai lebih besar dari jam mulai.

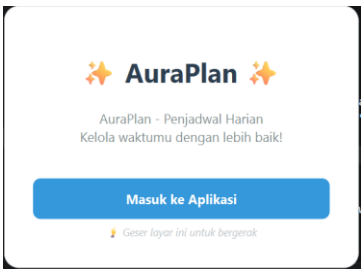
e. `Database` jika valid, memanggil fungsi `create(data)`. `Database` mengeksekusi perintah SQL INSERT INTO.

f. `Controller` setelah sukses, memanggil fungsi `load_data()` untuk mengambil data terbaru.

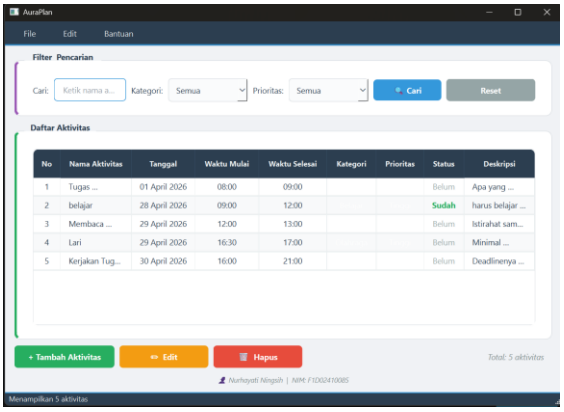
g. `Controller` meneruskan data baru tersebut ke fungsi `update_table()` di Main Window. `QTableWidget` langsung diperbarui secara visual.

6. Screenshoot antarmuka aplikasi

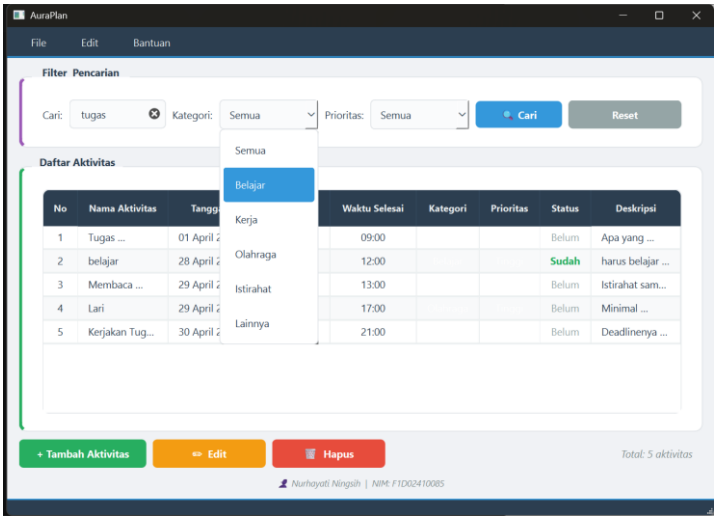
a. Tampilan Welcome



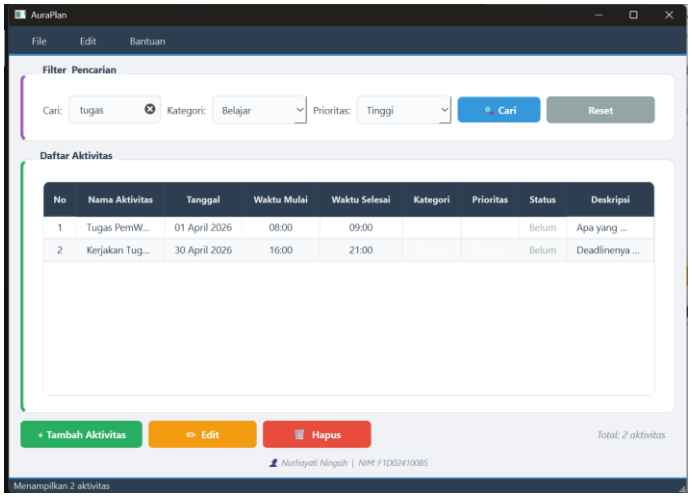
b. Tampilan Utama Aplikasi



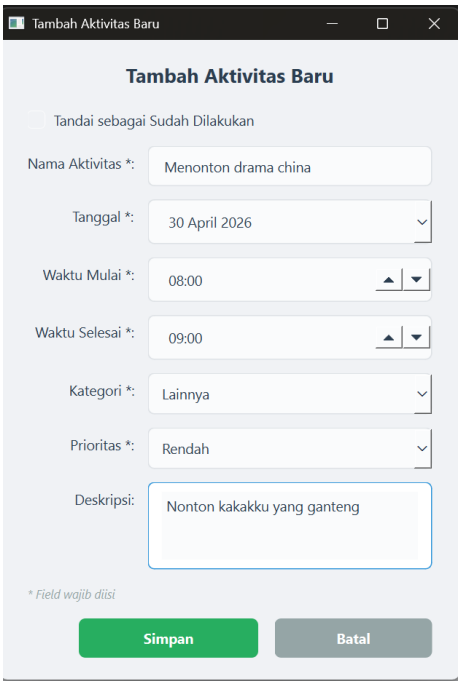
c. Tampilan filter pencarian dengan pilihan kategori



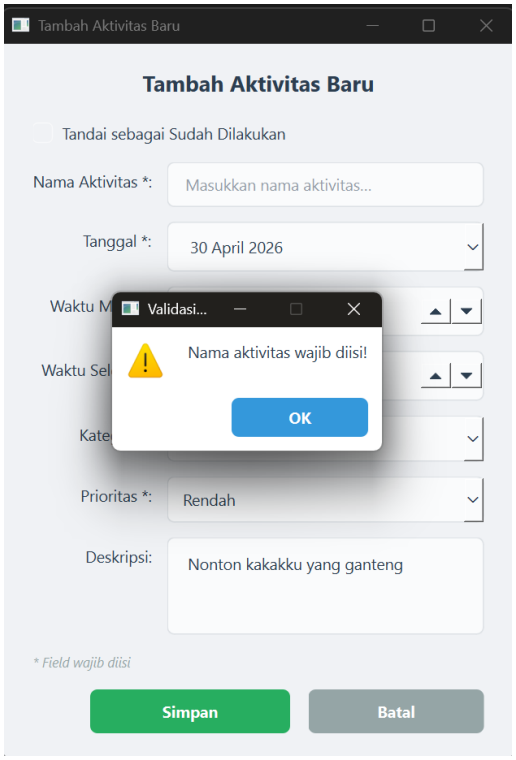
d. Tampilan Filter Pencarian Tekan Tombol Cari



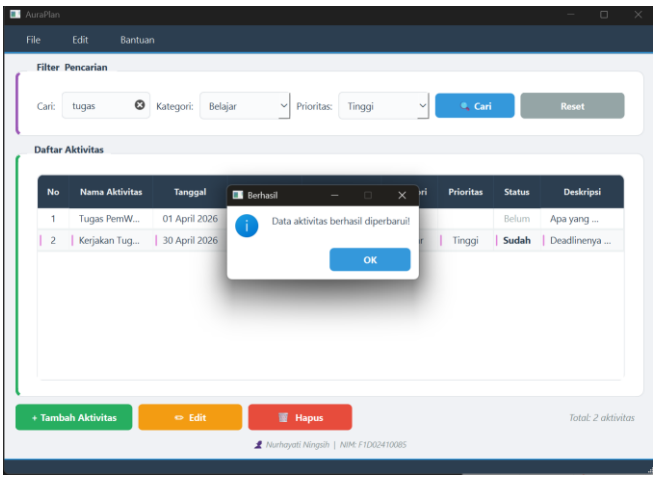
e. Tampilan Tambah Aktivitas Baru



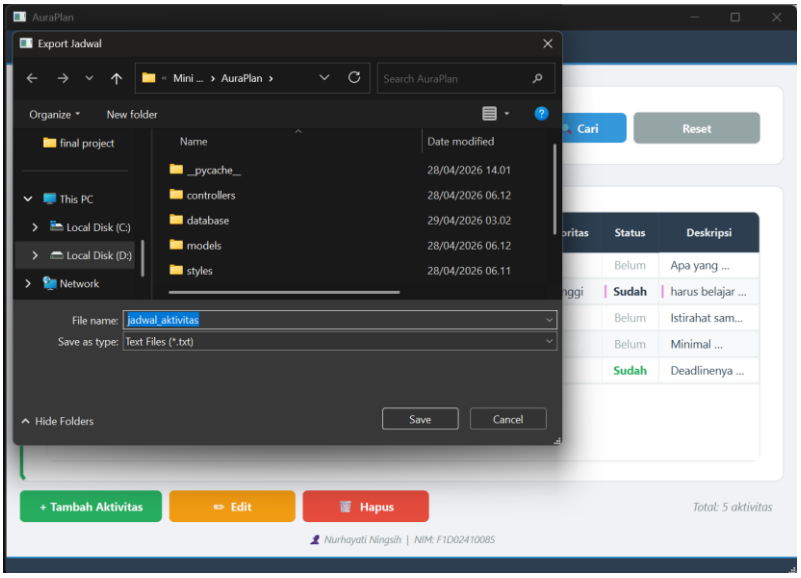
f. Tampilan Konfirmasi Wajib Diisi



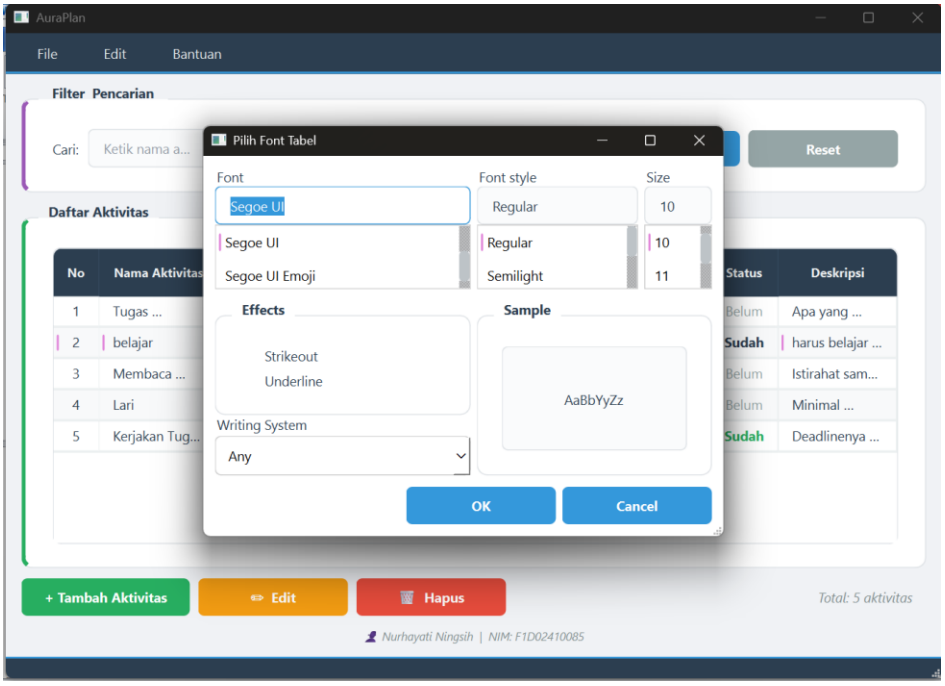
g. Tampilan Edit Aktivitas



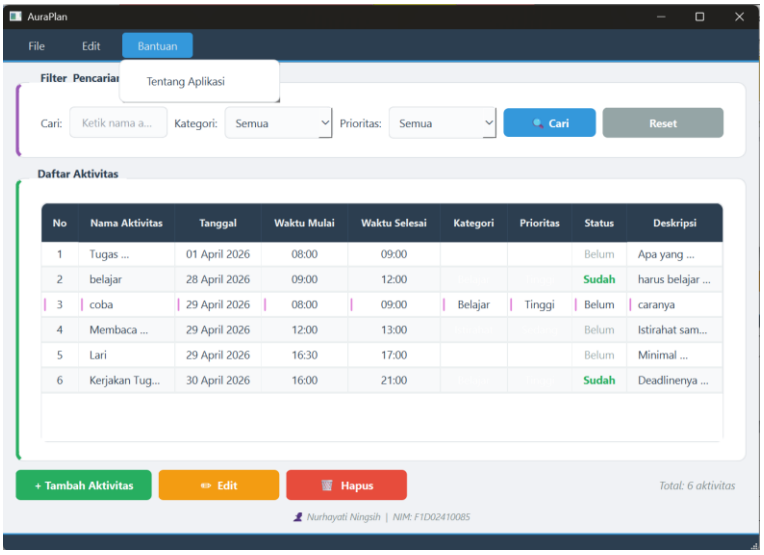
h. Tampilan Ingin Ubah Menjadi File TXT



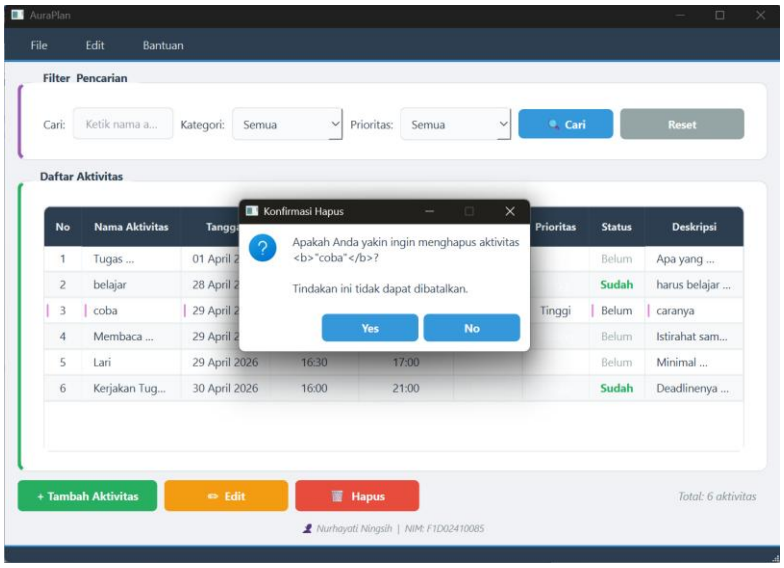
i. Tampilan Ingin Mengubah Font

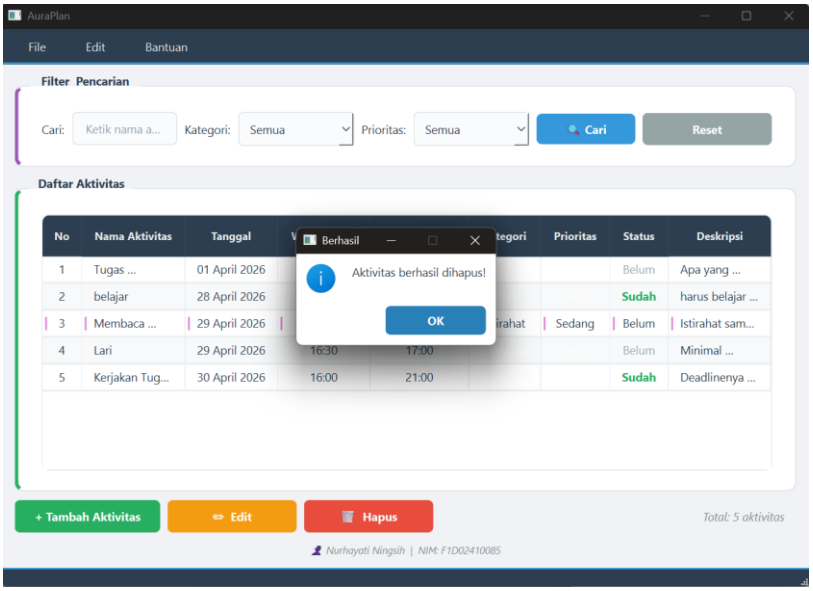


j. Tampilan Tombol Bantuan

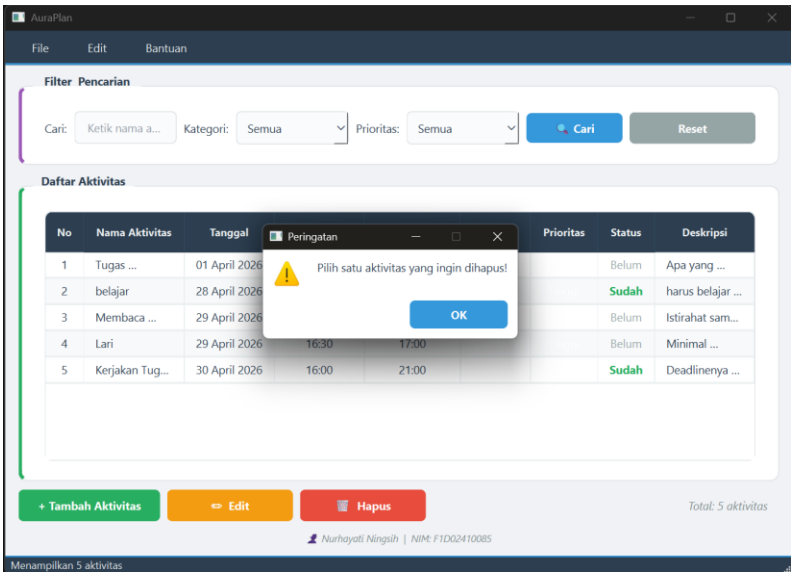


k. Tampilan Konfirmasi Hapus Jadwal

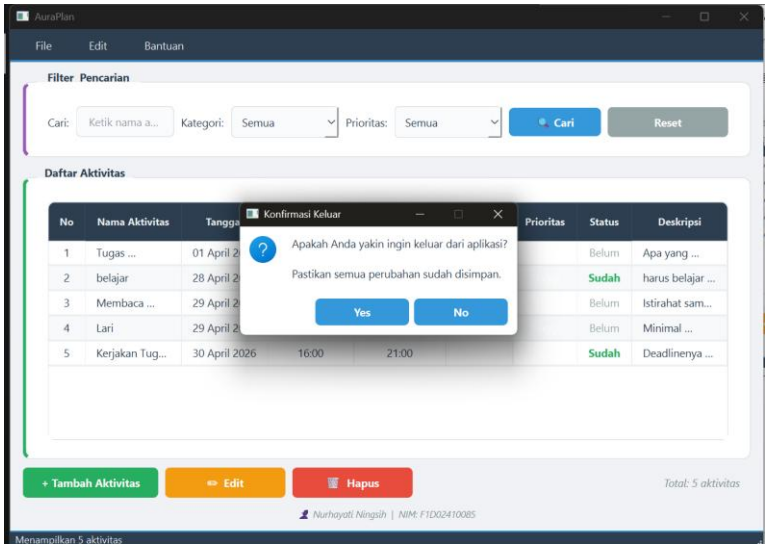




l. Tampilan Peringatan Jika Belum Memilih Jadwal yang Ingin di Hapus



m. Tampilan Validasi Keluar dari aplikasi Setelah Tekan [X]



7. **Link YouTube**

https://youtu.be/4PW_SrV31Yw?si=tE8vbsYrd0mC0La4

8. **Link Github**

https://github.com/nurhayatiningsih/pv26-miniproject-penjadwal-aktivitas-F1D02410085